

COMP 452 Assignment 1 Description File

Bryce Dixon (3397649) Jan 2026

TABLE OF CONTENTS

Part 1: Logic and Game Design - Page 1

Part 2: Game Compilation Instructions - Page 3

Part 3: Game Play Instructions - Page 4

Part 4: Implemented Steering Behaviour + How to Cause These Behaviours - Page 4

Part 5: Current Game Bugs/ Problems - Page 6

PART 1: Logic and Game Design

DESIGN GOAL:

My main goal with this game was to highlight and demonstrate the steering behaviours we have learned in this course thus far. I tried to leverage these behaviours to create a simple game which is enjoyable to play, and provides me ample practice with implementing and visualizing these behaviours.

CLASS STRUCTURE OVERVIEW:

- Main
 - Character
 - Player
 - ComputerPlayer
 - Collectables
 - SteeringBehaviourBlend
 - FollowBehaviourBlend
 - SeparationBehaviourBlend
 - CohesionBehaviourBlend
 - BlendedBehaviours
 - BehaviourSteerOutput
 - BehaviourAndWeight

MAIN GAME LOGIC:

- There is a user-controlled object and a computer controlled object, which compete to collect wandering objects.
- The collectable objects transition from a wandering behaviour to a specific steering behaviour when collected.
 - The specific steering behaviour depends on which player has collected it.
- When the game loads, the user and computer are spawned in the bottom right corner, while all collectable items are spawned randomly throughout the rest of the map (this is set up so that they do not spawn directly on top of the user and computer).

PLAYER LOGIC:

- When the player is within 100 pixels of a wandering object, the object locks onto the player and performs a seek/arrive behaviour.
- The player is controlled using arrow keys, all keys pressed are added to a Set, so multiple key presses can be employed simultaneously, allowing for diagonal movement as well.

COMPUTER LOGIC:

- The computer player finds the next closest, and available, wandering purple circle.
- When the computer is within 100 pixels of a wandering object, the object treats the computer as its target and performs a blended weighting of 3 behaviours (follow, separation, and cohesion).
 - I tuned this behaviour (modified weights and speeds) to look like a school of fish following the computer player.

GAME LOOP:

- My Main.java file holds the game loop which calls the update method of each game object (which updates the position of each object) and also handles the toroidal game map logic for each object.
 - All objects are simply in an ArrayList and the game continually loops through and updates these. This is why all game objects were set up to extend the Character class, as then they can all be in the same ArrayList<Character>.

GAME END:

- Once all wander objects have been collected, the game screen indicates if the user has won or lost (also visible by the score board in the top left corner).
- The computer also will lock onto the user as its target for a seek/arrive behaviour.
 - This was added as:
 - It is very easy to see all implemented behaviours by leading the computer and its collected items around.
 - It is also quite satisfying as well.

PART 2: Game Compilation Instructions

- I wasn't sure how much detail you wanted so I made it more of a step by step explanation.
- There are not any sounds, textures, or images in my game, I simply used JavaFX shapes.

IF USING JAVA 8:

- Since JavaFX is included with Java8, you can simply navigate to the directory where you have saved the jar file and run:
 - `java -jar A1_452_Dixon_3397649.jar`

IF USING JAVA 11 and HIGHER:

- This project was developed and tested on a MacOS device using Java 17, meaning I needed to download and install JavaFX. I used JavaFX version 21.0.9.
- To run the .jar file, save the file to your device in a known location.
- Ensure JavaFX is downloaded, and find the path to the JavaFX 'lib' folder.
- In your command line interface (CLI), Navigate to the location where you saved the .jar file.
 - Now we tell java what libraries to include from JavaFX, as well as where to find them (i.e. the path to "lib" mentioned above), and then run the program.
 - An example of this is:


```
desktop % java \
--module-path /Users/bd/Desktop/AU/COMP_452/javafx/lib \
--add-modules javafx.controls,javafx.graphics \
-jar A1_452_Dixon_3397649.jar
```

 - Where the file (A1_452_Dixon_3397649.jar) is stored on on my desktop, the path to my javaFX library is /Users/bd/Desktop/AU/COMP_452/javafx/lib, and we are using the modules javafx.controls,javafx.graphics from the JavaFX library to run the program.

PART 3: Game Play instructions

- The game starts on a pause screen.
 - The play/pause button is in the top right corner.
 - Press this to start when ready.
- The player and computer's scores are in the top left corner.
 - These increment as the players collect the collectable objects.
- The user player is a blue square controlled using the arrow keys.
- The computer is a red square which performs the behaviours indicated in PART 1 above.
- As previously mentioned, the objective is to collect more wandering purple circles than the computer player.
- The game map is also toroidal, allowing all objects in play to move fluidly through one edge of the screen and appear on the opposite edge.

PART 4: Implemented Steering Behaviours + How To Cause These Behaviours

- Implemented behaviours:
 - Basic Algorithms: Seek(), Arrive(), Wander()
 - Blended Algorithm: Weighted blending of follow(), separation(), cohesion()
- COLLECTABLE OBJECTS:
 - Perform a wandering behaviour (the same one described in the textbook) when they are not collected. This wandering occurs at a speed slightly faster than the computer and user move, to add a bit of strategy to catching them.
 - Once collected, these objects perform various behaviours:
 - COMPUTER: If collected by the computer, the collectable object performs a weighted blending of 3 behaviours (follow, separation, and cohesion). Follow has the weakest weight, as it has the most powerful acceleration vectors, while separation has the highest weighting, as it has weaker vectors typically. Cohesion has a smaller weighting as well, as cohesion is less of a concern due to the follow algorithm making the objects want to bunch up already. However, cohesion was added as it created a more natural movement when the computer object is stationary.
 - This behaviour blending design was based off of the logic described in our textbook regarding the flocking algorithm.
 - I modified each behaviour slightly:
 - Separation: I used the inverse square law to make it so that items which are closer push "harder" than neighbours further away. This was done as when all neighbours pushed the same amount, the flock was much more rigid and jittery. This also is partially why the values from separation were much smaller and why it needed an even larger weighting.

- Example code: `steerOutput.x += (diffX/distanceApart) * (1/distanceApart);`
 - Cohesion: this was added as when the computer is stationary, separation was too strong and needed an opposing force. This made the objects move much more smoothly when the computer was stationary.
 - Follow: this replaced "alignment" from the flocking algorithm, and was based off of a seek/ arrive algorithm, but adjusted to output accelerations for the weighting.
 - USER: If collected by the user player, the collectable items perform a seek/arrive behaviour with the user as their target. The radius of the arrive portion is slightly larger to more easily see this when the objects are close to the user/ when the user has stopped.
 - I made a few tweaks to the game to more easily see these behaviours:
 - When the collectable items are collected they change colour:
 - If caught by the computer: they turn a light red (as the computer object is red),
 - If caught by the user: they turn light blue (as the user object is blue).
 - When collected, these items also adjust their speed:
 - When caught by the computer, they increase in speed to exaggerate the blended behaviours more.
 - When caught by the user, they reduce their speed to more easily demonstrate the seek and arrive behaviours.
 - Each successive item collected is given an incrementally slower speed so that they space out when chasing the user.
- COMPUTER PLAYER
 - The computer player uses a modified seek and arrive algorithm to pursue the next closest collectable item.
 - Once all collectable items are collected, it uses this same algorithm to pursue the user.
 - The user can then be moved around to easily see the implemented behaviours of the computer player, and the collectable items of both the user and the computer.

PART 5: Current Game Bugs/ Problems

- I did not add a restart button as it was not fully resetting the scene and I was concerned it would impede gameplay.
 - If you would like to see this on the next assignment, I will implement it.
- I did not implement any audio, which could enhance gameplay if the game was to be built further.
- Since collectable objects are placed randomly throughout the map when the game loads, they can clump together and sometimes this can bias which player wins. I did make these random values not be over top of where the user and computer are spawned.
- If the computer gets a head start it can be very quick. This is why the computer is slightly slower than the user and starts slightly offset (behind the user). Unfortunately this then sometimes biases the user to win quickly. But due to the randomness of the game this is highly variable.
- The game is very quick, which I like but this may not be for everyone.
- The collectable items for the player do overlap once they are all collected and the player stops. This is not an issue, but it may be nice to offset them to see all the items collected. This is why when they are collected, each one has a slightly slower velocity, so they space out when chasing the user.
- The computer does not use the toroidal logic to its advantage (i.e. if it loses an object which has travelled out of bounds and spawned across the map, it will move on to a different object). This is usually the best way to beat the computer at the game and something I think is good. But it could be something to implement for more advanced gameplay.
- I have not run into any known crashes or performance issues when testing gameplay.